

Rules to Compute FIRST Set

1. If x is a terminal, then $\text{FIRST}(x) = \{ 'x' \}$
2. If $x \rightarrow \epsilon$, is a production rule, then add ϵ to $\text{FIRST}(x)$.
3. If $X \rightarrow Y_1 Y_2 Y_3 \dots Y_n$ is a production,
 1. $\text{FIRST}(X) = \text{FIRST}(Y_1)$
 2. If $\text{FIRST}(Y_1)$ contains ϵ then $\text{FIRST}(X) = \{ \text{FIRST}(Y_1) - \epsilon \} \cup \{ \text{FIRST}(Y_2) \}$
 3. If $\text{FIRST}(Y_i)$ contains ϵ for all $i = 1$ to n , then add ϵ to $\text{FIRST}(X)$.

Rules to compute FOLLOW set:

- 1) $\text{FOLLOW}(S) = \{ \$ \}$ // where S is the starting Non-Terminal
- 2) If $A \rightarrow pBq$ is a production, where p , B and q are any grammar symbols,
then everything in $\text{FIRST}(q)$ except ϵ is in $\text{FOLLOW}(B)$.
- 3) If $A \rightarrow pB$ is a production, then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.
- 4) If $A \rightarrow pBq$ is a production and $\text{FIRST}(q)$ contains ϵ ,
then $\text{FOLLOW}(B)$ contains $\{ \text{FIRST}(q) - \epsilon \} \cup \text{FOLLOW}(A)$

Example 1:

Production Rules:

```
E -> TE'
E' -> +T E' | ε
T -> F T'
T' -> *F T' | ε
F -> (E) | id
```

FIRST set

```
FIRST(E) = FIRST(T) = { ( , id }
FIRST(E') = { +, ε }
FIRST(T) = FIRST(F) = { ( , id }
FIRST(T') = { *, ε }
FIRST(F) = { ( , id }
```

FOLLOW Set

```
FOLLOW(E) = { $ , ) } // Note ')' is there because of 5th rule
FOLLOW(E') = FOLLOW(E) = { $, ) } // See 1st production rule
FOLLOW(T) = { FIRST(E') - ε } ∪ FOLLOW(E') ∪ FOLLOW(E) = { + , $ , ) }
FOLLOW(T') = FOLLOW(T) = { + , $ , ) }
FOLLOW(F) = { FIRST(T') - ε } ∪ FOLLOW(T') ∪ FOLLOW(T) = { *, + , $ , ) }
```

Example 2:

Production Rules:

$S \rightarrow aBDh$
 $B \rightarrow cC$
 $C \rightarrow bC \mid \epsilon$
 $D \rightarrow EF$
 $E \rightarrow g \mid \epsilon$
 $F \rightarrow f \mid \epsilon$

FIRST set

$\text{FIRST}(S) = \{ a \}$
 $\text{FIRST}(B) = \{ c \}$
 $\text{FIRST}(C) = \{ b, \epsilon \}$
 $\text{FIRST}(D) = \text{FIRST}(E) \cup \text{FIRST}(F) = \{ g, f, \epsilon \}$
 $\text{FIRST}(E) = \{ g, \epsilon \}$
 $\text{FIRST}(F) = \{ f, \epsilon \}$

FOLLOW Set

$\text{FOLLOW}(S) = \{ \$ \}$
 $\text{FOLLOW}(B) = \{ \text{FIRST}(D) - \epsilon \} \cup \text{FIRST}(h) = \{ g, f, h \}$
 $\text{FOLLOW}(C) = \text{FOLLOW}(B) = \{ g, f, h \}$
 $\text{FOLLOW}(D) = \text{FIRST}(h) = \{ h \}$
 $\text{FOLLOW}(E) = \{ \text{FIRST}(F) - \epsilon \} \cup \text{FOLLOW}(D) = \{ f, h \}$
 $\text{FOLLOW}(F) = \text{FOLLOW}(D) = \{ h \}$

Example 3:

Production Rules:

$S \rightarrow ACB \mid Cbb \mid Ba$
 $A \rightarrow da \mid BC$
 $B \rightarrow g \mid \epsilon$
 $C \rightarrow h \mid \epsilon$

FIRST set

$\text{FIRST}(S) = \text{FIRST}(A) \cup \text{FIRST}(B) \cup \text{FIRST}(C) = \{ d, g, h, \epsilon, b, a \}$
 $\text{FIRST}(A) = \{ d \} \cup \{ \text{FIRST}(B) - \epsilon \} \cup \text{FIRST}(C) = \{ d, g, h, \epsilon \}$
 $\text{FIRST}(B) = \{ g, \epsilon \}$
 $\text{FIRST}(C) = \{ h, \epsilon \}$

FOLLOW Set

$\text{FOLLOW}(S) = \{ \$ \}$
 $\text{FOLLOW}(A) = \{ h, g, \$ \}$
 $\text{FOLLOW}(B) = \{ a, \$, h, g \}$
 $\text{FOLLOW}(C) = \{ b, g, \$, h \}$

Note:

1. ϵ as a FOLLOW doesn't mean anything (ϵ is an empty string).
2. $\$$ is called end-marker, which represents the end of the input string, hence used while parsing to indicate that the input string has been completely processed.
3. The grammar used above is [Context-Free Grammar](#) (CFG). The syntax of a programming language can be specified using CFG.
4. CFG is of the form $A \rightarrow B$, where A is a single Non-Terminal, and B can be a set of grammar symbols. (i.e. Terminals as well as Non-Terminals)

Features of FOLLOW Set

The FOLLOW set in syntax analysis has several important features that make it essential for parsing algorithms, especially in predictive parsers like LL(1). Here are its key features:

1. **Identifies Valid Successors:** It contains all terminal symbols that can immediately follow a non-terminal in any derivation.
2. **End-of-Input Marker ($\$$):** For the start symbol of a grammar, the FOLLOW set always includes the end-of-input marker $\$$, indicating the end of parsing.
3. **Assists in Parsing Table Construction:** It is used in [LL\(1\)](#) parsing to decide which production rule to apply based on the next input symbol.
4. **Handles Nullable Productions:** If a non-terminal can derive an empty string (nullable), the FOLLOW set of that non-terminal contributes to its parent non-terminal.
5. **Ensures Predictive Parsing:** By using the FOLLOW set, parsers avoid ambiguity and ensure that only one rule is chosen for a given input.
6. **Closely Linked to FIRST Set:** The FOLLOW set works with the [FIRST set](#) to resolve grammar rules and predict parsing steps effectively.